LLM Systems in Practice: Applications, Economics & Case Studies

IDS 576: Deep Learning and Modern Applications

Dr. Theja Tulabandhula

University of Illinois Chicago

Learning Objectives

Learning Objectives

After this lecture, you will be able to:

- Explain why RAG is necessary for production LLM applications
- Understand how text embeddings enable semantic search
- Compare different vector database options and design a RAG pipeline
- Trace the path from user input to ChatGPT output
- Explain how training, safety systems, and tools create conversational ability

Learning Objectives (cont.)

- Explain the major cost components of LLMs (training + inference)
- Compare different business models for LLM providers
- Explain how Mixture-of-Experts improves efficiency
- Understand DeepSeek's RL approach to reasoning
- Analyze why DeepSeek was considered disruptive

Part I: Retrieval-Augmented Generation (RAG)

Introduction to RAG

Why RAG?

LLMs have inherent limitations:

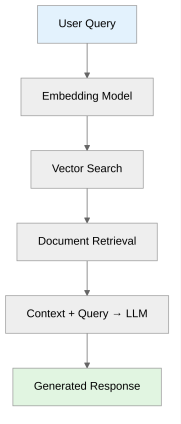
- Knowledge cutoff (training data is static)
- Hallucinations (generating plausible but incorrect information)
- No access to private/proprietary data
- Limited context window (cannot fit all relevant information)

RAG addresses these by:

- Retrieving relevant documents from external sources
- Augmenting prompts with retrieved context
- Grounding responses in factual information
- Enabling access to updated and private knowledge

Core idea: Combine LLM reasoning with external knowledge retrieval

RAG Architecture



RAG Key Components

1. Document embedding and indexing
2. Vector database for similarity search
3. Retrieval mechanism
4. LLM generation with augmented context

Embeddings

Text Embeddings

Definition: Dense vector representations of text that capture semantic meaning

Example:

```
"machine learning" → [0.42, -0.13, 0.88, ..., 0.15]
"artificial intelligence" → [0.39, -0.11, 0.85, ..., 0.18]
```

Key properties:

- Similar texts have similar embeddings (high cosine similarity)
- Dimensionality typically 384-1536 for modern models
- Fixed-size regardless of input text length

Popular Embedding Models

- `sentence-transformers` (all-MiniLM-L6-v2, all-mpnet-base-v2)
- OpenAI `text-embedding-3-small/large`
- `Cohere embed-english-v3.0`

Similarity Search

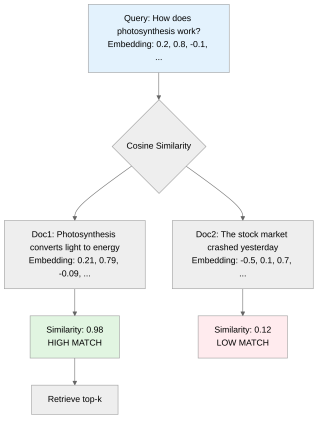
How to find relevant documents:

Cosine similarity:

```
similarity = (A · B) / (||A|| × ||B||)
```

Similarity Search Illustration

Example:



Result: Retrieve top-k most similar documents

Creating Embeddings: Indexing

Offline indexing process:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2') # Load embedding model

# Embed documents
documents = [
    "Transformers use attention mechanisms",
    "RAG combines retrieval with generation",
    "Vector databases enable similarity search"]

embeddings = model.encode(documents) # Store in vector database
```

Creating Embeddings: Querying

Online query process:

```
query = "How does attention work?"
query_embedding = model.encode([query])
# Search vector database
```

Vector Databases

What is a Vector Database?

Specialized database for storing and searching embeddings

Key features:

- Efficient approximate nearest neighbor (ANN) search
- Horizontal scalability for billions of vectors
- Metadata filtering (combine vector search with traditional filters)
- CRUD operations for embeddings

Why Vector Databases?

Why not use traditional databases?

- Exact nearest neighbor is O(n) - too slow for large collections
- Need specialized indexes (HNSW, IVF) for fast ANN search
- Vector databases optimize for high-dimensional similarity search

Popular Vector Databases (2024-2026)

Database	Type	Key Features
`Pinecone`	Cloud	Managed, serverless, production-ready
`Weaviate`	OSS/Cloud	Hybrid search, multi-modal
`Qdrant`	OSS/Cloud	Rust-based, high performance
`FAISS`	Library	CPU/GPU, research-focused
`Chroma`	OSS	Lightweight, Python-native
`Milvus`	OSS/Cloud	Distributed, large-scale

Selection criteria:

- Scale (millions vs billions of vectors)
- Latency requirements
- Cloud vs self-hosted
- Integration with existing infrastructure

Example: Qdrant Setup

```python
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams

# Initialize client
client = QdrantClient(":memory:")

# Create collection
client.create_collection(
    collection_name="documents",
    vectors_config=VectorParams(size=384, distance=Distance.COSINE)
)
```

Example: Qdrant Insert and Search

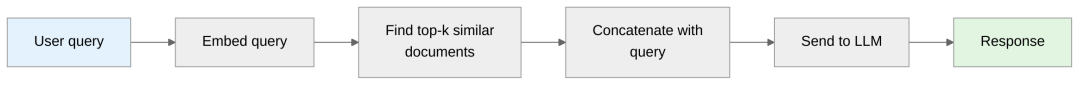
```
# Insert vectors
client.upsert(
    collection_name="documents",
    points=[
        {"id": 1, "vector": embedding1, "payload": {"text": doc1}},
        {"id": 2, "vector": embedding2, "payload": {"text": doc2}}
    ]
)

# Search
results = client.search(
    collection_name="documents",
    query_vector=query_embedding,
    limit=5)
```

Retrieval Strategies

Basic RAG Pipeline

Simple retrieval:



Example prompt:

```
Context:
[Retrieved Document 1]
[Retrieved Document 2]
[Retrieved Document 3]

Question: {user_query}

Answer based on the context above:
```

Basic RAG: Limitations

- May retrieve irrelevant documents
- No re-ranking of results
- Static chunk size

Advanced Retrieval

Hybrid search: Combine vector similarity with keyword search

```
Final_Score = α × Vector_Score + (1-α) × Keyword_Score
```

Hypothetical Document Embeddings (HyDE):

1. Generate hypothetical answer using LLM
2. Embed the generated answer
3. Search using answer embedding (better semantic match)

Multi-Query and Implementation

Multi-query retrieval:

1. Generate multiple variations of user query
2. Retrieve documents for each variation
3. Combine and deduplicate results

Repository: `langchain` (2026, clickable) provides implementations

Chunking Strategies

Problem: Documents may be too long for embedding models (512-1024 token limits)

Solution: Split documents into chunks

Strategies:

```
Fixed-size chunks:
"The transformer architecture..." (500 tokens)
"...self-attention mechanism..." (500 tokens)

Semantic chunks:
Split on paragraphs or section boundaries

Overlapping chunks:
Include 50-100 token overlap for continuity
```

Chunking Trade-offs

- Smaller chunks: More precise retrieval, less context
- Larger chunks: More context, less precise retrieval

Best practice: 300-500 tokens with 50 token overlap

RAG Evaluation

Key Metrics

Retrieval quality:

- **Precision@k:** What fraction of top-k results are relevant?
- **Recall@k:** What fraction of relevant documents are in top-k?
- **Mean Reciprocal Rank (MRR):** Position of first relevant document

Generation quality:

- **Faithfulness:** Does answer align with retrieved context?
- **Answer relevance:** Does answer address the question?
- **Context relevance:** Are retrieved documents relevant to query?

End-to-End and Evaluation Frameworks

End-to-end:

- **Accuracy:** Correctness of final answer
- **Latency:** Time from query to response

Evaluation Frameworks

RAGAS (RAG Assessment):

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevance,
    context_relevance
)

results = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevance, context_relevance]
)
```

Evaluation Frameworks (cont.)

Manual evaluation:

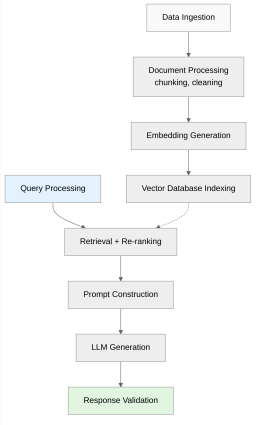
- Human annotation of answer quality
- Side-by-side comparisons (RAG vs non-RAG)
- Domain expert review for factual accuracy

Repository: ragas (2026, clickable)

Production RAG

RAG System Design

Complete production pipeline:



RAG System Design (cont.)

Additional components:

- Caching (query and document caches)
- Monitoring and logging
- Fallback mechanisms
- Access control and security

Challenges and Solutions

Challenge	Solution
Stale data	Periodic re-indexing, incremental updates
Long documents	Hierarchical retrieval, summarization
Multi-lingual	Multi-lingual embedding models
Query ambiguity	Query expansion, clarification prompts
Cost	Cache common queries, batch processing

Production RAG Frameworks (2024-2026)

- `LlamaIndex`: Data framework for LLM applications
- `LangChain`: Application framework with RAG primitives
- `Haystack`: NLP framework with RAG support

Part II: How ChatGPT (2024-2025) Actually Works

The ChatGPT Pipeline

Overview

What happens when you type a prompt into ChatGPT?

Complete pipeline from input to output:

`User Prompt → Tokenization → Transformer → Generation → Safety → Response`

The Big Picture

Modern chat assistants combine several systems:

- **LLM:** Foundation model (GPT architecture)
- **Training:** Pretraining + SFT + RLHF
- **Inference:** Optimized serving infrastructure
- **Safety:** Input/output moderation
- **Tools:** Web search, code execution, retrieval

ChatGPT = `model + training + systems + safety`

Real-Time Inference Pipeline

Step 1: User Prompt

User inputs: `"Explain how transformers work"`

System adds formatting:

```
<|system|>You are a helpful AI assistant.<|end|>
<|user|>Explain how transformers work<|end|>
<|assistant|>
```

Step 1: Additional Context

- Conversation history
- System instructions
- User preferences

Step 2: Tokenization

Text → token IDs using BPE

Example:

```
"ChatGPT works well"
    ↓
["Chat", "G", "PT", " works", " well"]
    ↓
[1273, 38, 11571, 4375, 1664]
```

Token vocabulary: ~50,000-100,000 tokens

Special tokens: `<|endoftext|>`, `<|im_start|>`, etc.

Step 3: Embedding Layer

Token IDs → vectors:

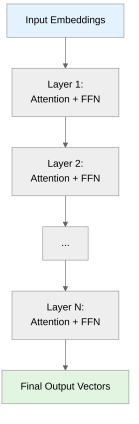
```
[1273, 38, 11571, ...]
    ↓
[[0.31, -0.82, 0.15, ...],
 [0.42, 0.13, -0.28, ...],
 ...]
```

Add positional encodings to preserve order

Feed into transformer stack

Step 4: Transformer Processing

Stack of decoder blocks process tokens:



Each token attends only to previous tokens (causal masking)

Step 5: Next Token Prediction

Final layer outputs probability distribution:

```
Linear projection + Softmax
    ↓
P(next token | context)
```

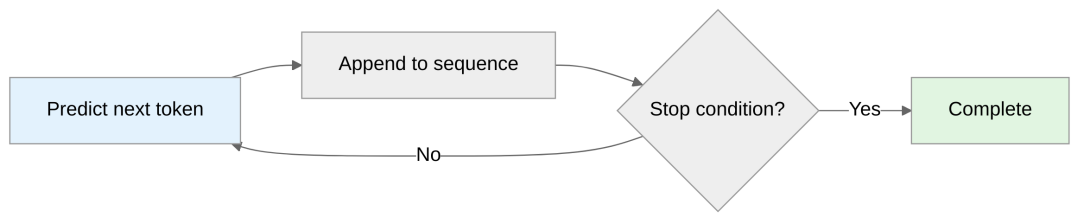
Step 5: Sampling Strategies

```
Example output probabilities:
"Transformers" → 0.52
"Neural"       → 0.19
"Attention"    → 0.11
...
```

Sampling strategies: greedy, top-k, top-p, temperature

Step 6: Autoregressive Generation

Generation loop:



Example:

```
"Explain transformers" → [generates] "Transformers"
"...Transformers" → [generates] "are"
"...are" → [generates] "neural"
...
```

Uses KV cache for efficiency

Step 7: Safety Moderation

Output passes through moderation layers:

Content filters:

- Harmful content detection
- PII (Personal Identifiable Information) removal
- Toxicity screening

Safety: Refusal Behavior

```
If unsafe request detected:
    Return refusal message
Else:
    Return generated text
```

Safety is applied at both input and output

Offline Training Pipeline (Recap)

Training Pipeline Summary

The three-phase pipeline:

```
Pretraining → Instruction Tuning (SFT) → RLHF → Deployed Model
```

Phase 1: Pretraining: Learn language patterns from ~1-10 trillion tokens

Phase 2: SFT: Train on 10k-100k high-quality (prompt, response) pairs

Phase 3: RLHF: Align with human preferences via reward model + PPO

Alignment Goals

RLHF optimizes for:

Helpful:

- Answers questions accurately
- Follows instructions
- Asks clarifying questions when needed

Harmless:

- Refuses harmful requests
- Avoids biased/toxic output
- Respects privacy

Honest:

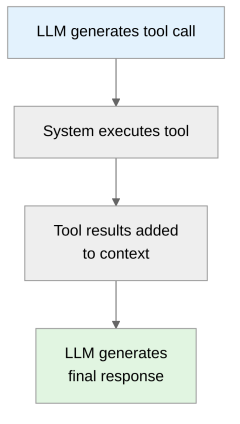
- Admits uncertainty
- Doesn't fabricate information
- Cites limitations

Advanced Capabilities

Tool Use

Modern assistants can use external tools:

Process:

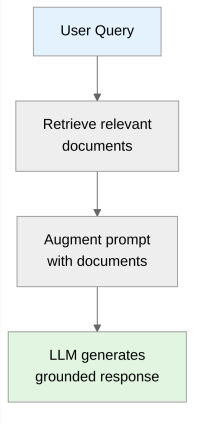


Common Tools

- Web search (`Bing`, Google)
- Code interpreter (Python execution)
- Calculator
- Calendar/scheduling

Retrieval Augmented Generation in ChatGPT

ChatGPT can access external knowledge:



Retrieval Augmented Generation in ChatGPT (cont.)

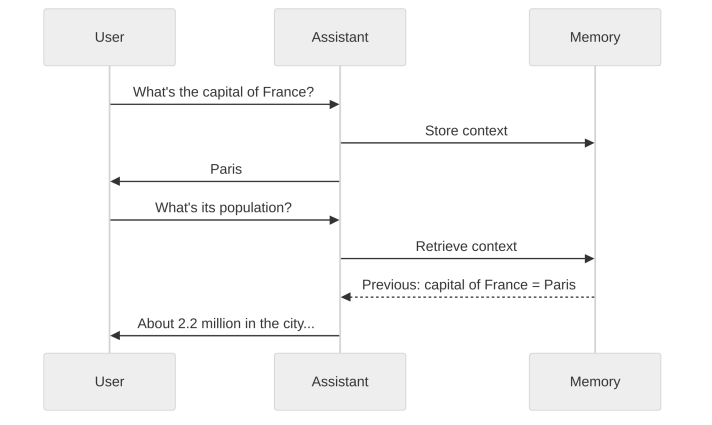
Benefits:

- Up-to-date information
- Access to user's data
- Reduced hallucinations

See Part I for detailed RAG coverage

Multi-turn Conversations

Maintaining context across turns:



Multi-turn Implementation

- Concatenate conversation history
- Truncate if exceeds context window
- Summarize very long conversations

ChatGPT (2024-25)

Infrastructure

Serving Architecture

Production system components:

```
Load Balancer
    ↓
API Gateway
    ↓
Model Servers (vLLM)
    ↓
GPU Clusters
    ↓
Monitoring & Logging
```

Serving Architecture (cont.)

Key challenges:

- High concurrency (millions of users)
- Low latency requirements
- GPU cost optimization

Inference Optimization

GPU Memory:

- KV cache with PagedAttention
- Quantization (FP16/FP8)

Throughput and Cost Optimization

Throughput:

- Continuous batching
- Speculative decoding

Cost:

- Caching common responses
- Model distillation for simple queries

Result: 10-20$\times$ cost reduction vs naive implementation

Scaling Challenges

As usage grows:

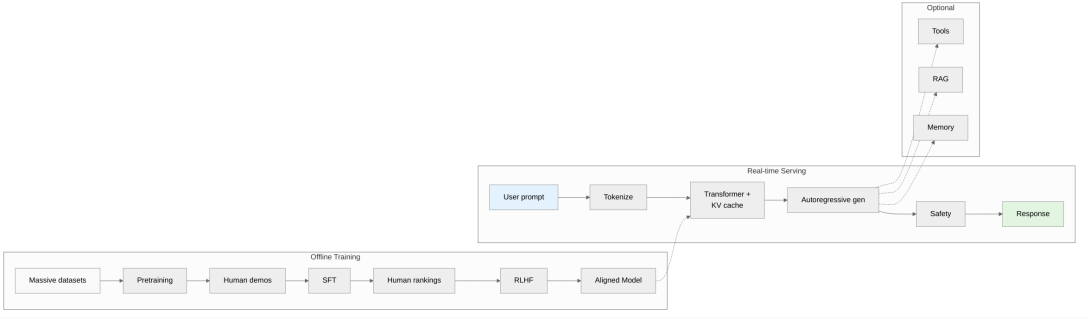
- Need more GPUs $\rightarrow$ supply constraints
- Higher costs $\rightarrow$ need better optimization
- Longer context $\rightarrow$ memory pressure
- More safety risks $\rightarrow$ need better moderation

Scaling Solutions

- Mixture-of-Experts models (activate subset of parameters)
- Distilled smaller models for simple tasks
- Edge deployment for low-latency use cases

The Complete ChatGPT System

Putting It All Together



What Makes ChatGPT Different: Training

Compared to base GPT models:

- Extensive RLHF for helpfulness
- Refusal training for safety
- Multi-task instruction tuning

What Makes ChatGPT Different: Systems

Systems:

- Highly optimized inference
- Tool integration
- Conversation management

Safety:

- Input/output moderation
- Adversarial testing
- Ongoing monitoring

Think of ChatGPT as an entire system that supports the deep learning model at its core

Part III: The Economics of Large Language Models

LLM Economics Introduction

Why Study LLM Economics?

LLMs are expensive to build and deploy

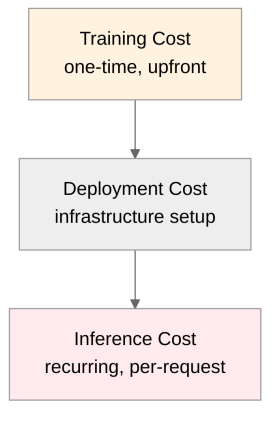
Understanding the economics explains:

- Why AI companies raise billions in funding
- Why NVIDIA GPUs are in high demand
- Why model pricing varies dramatically
- Why DeepSeek's low-cost approach was disruptive

Three major cost layers:

1. Training costs (one-time, upfront)
2. Inference costs (ongoing, per-request)
3. Infrastructure and operations

The AI Cost Stack



Key insight: Training is expensive once, inference is expensive forever

For high-volume services, **inference dominates total cost**

Training Costs

Components of Training Cost

Major drivers:

1. **GPU compute:** Thousands of GPUs for weeks/months
2. **Data preparation:** Cleaning, filtering, deduplication
3. **Engineering labor:** Research, experiments, debugging
4. **Experimentation:** Multiple training runs to find optimal setup

Example Training Cost

Example calculation for 7B parameter model:

```
100 GPUs × $2/hour × 720 hours = $144,000
+ Data costs + Engineering labor
= $200,000 to $500,000 total
```

Reported Training Costs (2025)

Model	Parameters	Reported/Estimated Cost	Source
GPT-3	175B	$4-12M	Industry estimates
GPT-4	Undisclosed	$78-100M+ (estimated)	Analyst estimates, unverified
Gemini Ultra	Undisclosed	$100-200M (estimated)	Analyst estimates, unverified
LLaMA 2 70B	70B	$5M+ (estimated)	Industry calculations
DeepSeek-V3	671B MoE	$5-6M (reported)	Company disclosure

Important: Most figures are industry estimates or calculations, not official disclosures. Actual costs vary based on infrastructure, failures, and methodology.

Trend: Costs growing as models scale, but efficiency improving

Why Training Is Expensive

Scaling laws drive costs:

```
Better performance = Larger models + More data + More compute
```

GPU requirements grow exponentially:

- 10× larger model needs 10× more GPUs or 10× longer training
- Frontier models use 10,000+ GPUs simultaneously

Example: Frontier Training Setup

```
16,000 NVIDIA A100 GPUs
Running for 6 weeks
At $2-3/GPU-hour
= $32-48M in compute alone
```

Additional costs: Power, cooling, networking, storage

Training vs Inference Trade-off

Key decision: Invest more in training for better inference efficiency

Options:

1. **Quick cheap training** → Larger model needed → Higher inference cost
2. **Expensive training** → Better optimization → Lower inference cost

Example:

- Undertrained 175B model at $10M training cost
- vs Optimally trained 70B model at $20M training cost
- The 70B may be cheaper long-term due to lower inference costs

Distillation: Train large teacher, use to train small student

Inference Costs

Why Inference Is Expensive

Each token generation requires:

- Full transformer forward pass
- Attention computation across all previous tokens
- Memory bandwidth for model weights and KV cache

GPU Utilization Challenge

Inference is sequential: Cannot parallelize token generation within a single request

- Training: Batch size 1000+ $\rightarrow$ high GPU utilization
- Inference: Often serial $\rightarrow$ lower utilization without batching

Cost Per Token

Typical API pricing (early 2025):

Provider	Model	Input (per 1M tokens)	Output (per 1M tokens)
OpenAI	GPT-4 Turbo	$10	$30
OpenAI	GPT-3.5 Turbo	$0.50	$1.50
Anthropic	Claude 3 Opus	$15	$75
Anthropic	Claude 3 Haiku	$0.25	$1.25
Google	Gemini Pro	$0.50	$1.50

Output costs more: Generation requires sequential processing

Trend: Prices decreasing over time due to optimization

Inference Cost Calculation

Example: Chatbot with 1M daily active users

```
Assumptions:
- 10 messages/user/day
- 100 tokens input + 200 tokens output per message
- Using GPT-3.5 Turbo pricing
```

Cost Calculation Result

```
Daily cost:
1M users × 10 messages × (100 input tokens + 200 output tokens on average)
× ($0.50/M input tokens + $1.50/M output tokens)
= $3,500/day = $1.27M/year
```

At scale, inference costs dominate

The Idle GPU Problem

Challenge: GPUs cost money even when idle

Scenarios:

```
Wait for requests → GPU idle → Wasted cost
Process immediately → High latency → Poor UX
```

Solutions:

- **Batching:** Group requests (but adds latency)
- **Auto-scaling:** Spin up/down GPUs based on demand
- **Multi-tenancy:** Share GPUs across customers

Tradeoff: Cost vs latency vs utilization

Optimization Strategies

Reducing Inference Cost

Techniques applied at scale:

1. Quantization:

```
FP16 → FP8: 2× cost reduction
FP16 → INT8: 2× cost reduction
FP16 → INT4: 4× cost reduction
```

2. Efficient architectures:

```
Mixture-of-Experts: Activate 10-20% of parameters
Smaller models for simple queries (cascade)
```

Caching and Optimization

3. Caching:

```
Cache common queries/responses
Share KV cache across similar requests
Prefix caching for repeated context
```

Cost Optimization Example

Baseline: GPT-4 class model, FP16, no optimization

Optimizations:

```
1. FP8 quantization → 2× reduction
2. KV cache optimization → 1.5× reduction
3. Continuous batching → 2× reduction
4. Response caching (10% hit rate) → 1.1× reduction

Total: 6.6× cost reduction
```

Result: $10M/year inference cost → $1.5M/year

Investment required: Engineering time + optimization infrastructure

GPU Economics

The GPU Bottleneck

NVIDIA currently dominates datacenter AI compute.

Approximate market pricing varies widely by channel and form factor: - A100 80GB: secondary market, often high four to low five figures - H100 80GB: often mid five figures - B200 / GB200-class Blackwell systems: newer and typically much more expensive; quote-based pricing is common

GPU Supply and Cloud Pricing

Cloud GPU pricing (on-demand):

- A100: $2-4/hour
- H100: $4-8/hour

Supply constraints:

- Lead times: 6-12 months for large orders
- Limited production capacity
- Export controls restrict availability

GPU Utilization Economics

Key metric: Revenue per GPU-hour

Example calculation:

```
GPU cost: $3/hour
Tokens generated: 50,000/hour (with batching)
Cost per token: $0.00006

To be profitable:
Need to charge > $0.00006/token
Typical charge: $0.0002/token
Margin: ~70%
```

Margin Considerations

Actual margins depend on:

- Utilization rate (idle time reduces effective margin)
- Engineering and operations costs
- Amortized training costs

Open vs Closed Model Economics

Closed models (OpenAI, Anthropic):

```
+ Higher prices
+ Better models (more training investment)
- API lock-in
- Less control
```

Open models (LLaMA, Mistral):

```
+ Self-hosting option
+ No API costs (after infrastructure)
+ Full control
- Need ML infrastructure expertise
- Upfront GPU costs
```

Break-even analysis: Self-hosting worth it at ~100M+ tokens/month

Business Models

LLM Provider Business Models

1. API Usage (consumption-based):

```
Pricing: Per token (input + output)
Examples: OpenAI, Anthropic, Cohere
Revenue scales with usage
```

Additional LLM Business Models

2. Subscriptions:

Pricing: Fixed monthly fee (ChatGPT Plus, $20/month)

Examples: ChatGPT Plus, Claude Pro

Predictable revenue, caps costs

3. Enterprise Licensing:

Pricing: Annual contracts ($100K-$10M+)

Examples: Azure OpenAI, custom deployments

Includes: SLAs, dedicated capacity, support

4. Hybrid:

Free tier → Subscription → Enterprise

Maximize total addressable market

The AI Cost Equation

For AI companies:

```
Revenue = Token usage × Price per token
Cost = GPU hours × Cost per GPU hour
Profit = Revenue - Cost - Overhead

Profitability requires:
1. High GPU utilization
2. Efficient inference (low cost per token)
3. Pricing power (high revenue per token)
```

Current Market State (2025)

- Some LLM providers not profitable yet
- Heavy investment in infrastructure
- Racing to achieve scale and efficiency

Why Some Models Are Free

Strategies for offering free access:

1. **Subsidized by paid tiers:** Free users drive adoption
2. **Research/marketing:** Build brand, gather data
3. **Open source:** Community development, ecosystem play
4. **Platform lock-in:** Free API to drive cloud sales

Examples:

- ChatGPT free tier: Marketing for Plus/Enterprise
- Google Gemini: Drive GCP adoption
- Meta LLaMA: Open source to establish ecosystem

Market Dynamics

The Compute Arms Race

Current trend: Massive infrastructure investment

Major players:

```
Anthropic: Raised billions
OpenAI: Raised billions
Google: Internal compute resources + TPUs
Meta: ...
```

Why the Compute Race?

- Compute = competitive moat
- First-mover advantage in AI
- Lock-in through ecosystem

Future Cost Trends: Optimistic

Better algorithms reduce compute needs:

- Hardware improvements (B200, TPUs)
- Competition drives prices down
- AI becomes commodity like cloud computing

Future Cost Trends: Pessimistic

- Frontier models require ever more compute
- GPU supply constraints persist
- Oligopoly pricing (few providers)
- High barriers to entry remain

Part IV: DeepSeek Case Study

DeepSeek Introduction

Why Study DeepSeek?

DeepSeek-V3 (December 2024) and DeepSeek-R1 (January 2025) drew attention because they challenged common assumptions:

1. Frontier-level models always require the very largest GPU clusters
2. Strong reasoning always requires large human-written reasoning datasets
3. Only a few US labs can ship models near the frontier

Why Study DeepSeek? (cont.)

Reported achievements (based on DeepSeek papers/model cards):

- DeepSeek-V3: 671B total parameters with 37B activated per token
- DeepSeek-V3 report claims only 2.788M H800 GPU hours for full training
- DeepSeek-R1 report claims reasoning performance comparable to OpenAI o1-1217 on some benchmarks
- Released weights plus relatively detailed technical reports for a frontier-adjacent model

Impact: market volatility, industry re-evaluation, and more attention to efficiency-focused model design

Note: Many headline cost/performance claims come from DeepSeek's own reports and should be treated as reported claims, not independently established facts

The DeepSeek Model Family

Timeline:

- **DeepSeek-V2** (May 2024): MoE model introducing MLA + DeepSeekMoE ideas
- **DeepSeek-V3** (December 2024): 671B-parameter MoE with 37B active parameters
- **DeepSeek-R1** (January 2025): reasoning-focused model post-trained from DeepSeek-V3-Base

Key innovation areas:

1. Mixture-of-Experts architecture
2. Reinforcement learning for reasoning
3. Cost-efficient training strategies
4. Hardware-aware optimization

DeepSeek Architecture Innovation

Mixture-of-Experts (MoE) in DeepSeek

Traditional dense transformer:

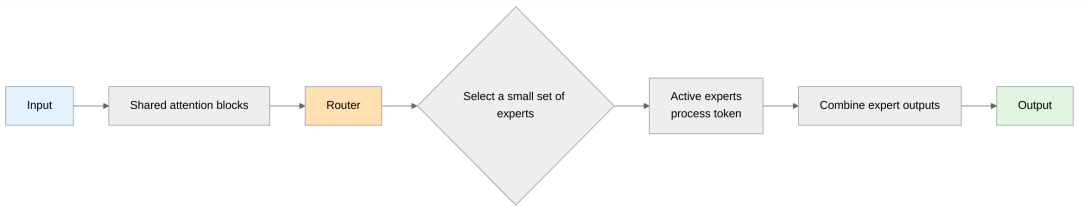
```
All parameters are used for every token
175B params -> roughly 175B parameter usages per token
```

MoE transformer:

```
Router selects only a few experts
671B total params -> ~37B active per token
```

MoE vs Dense Approach

DeepSeek-V3 architecture:



Why MoE Matters

Key intuition:

```
Very large total parameter count
Only a small subset active per token
=> more capacity at roughly dense-model active compute
```

MoE Efficiency Gains

Training efficiency:

- Smaller active subnetwork -> lower active compute than an equivalently sized dense model
- Less memory/bandwidth pressure per token than activating all parameters
- Better scaling when routing works well

Inference efficiency:

- Lower cost per token than a dense model with the same total parameter count
- Higher capacity without activating the full model every step

Trade-off: more complex routing, expert imbalance risk, and harder distributed systems engineering

MoE Implementation Details

Expert routing strategy:

```
# Simplified routing logic
router_scores = Router(hidden_state)
top_k_experts = TopK(router_scores, k=K)

output = 0
for expert_id in top_k_experts:
    weight = router_scores[expert_id]
    output += weight * Expert[expert_id](hidden_state)
```

MoE: Load Balancing and Comparison

Load balancing:

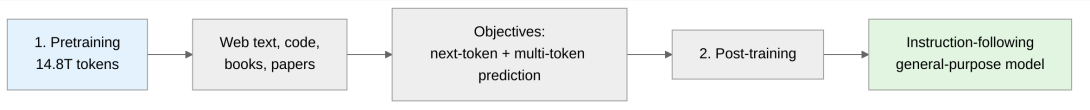
- The router must avoid sending most tokens to the same experts
- DeepSeek-V3 emphasizes auxiliary-loss-free load balancing
- Good balancing is critical for throughput and training stability

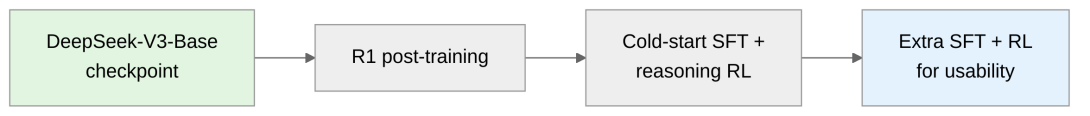
Aspect	Dense 70B	MoE 671B (DeepSeek-V3)
Active params	70B	37B
Total capacity	70B	671B
Approx. active compute	70B	~37B
Model quality	Strong baseline	Depends on routing/training quality

DeepSeek Training Strategy

Multi-Stage Training Pipeline

DeepSeek family training pipeline:





The first diagram summarizes the path to the general-purpose DeepSeek-V3 model, while the second shows the separate R1 pipeline starting from `DeepSeek-V3-Base`, not from the post-trained V3 instruct model.

Training Time and Efficiency

Reported training budget: 2.788M H800 GPU hours for DeepSeek-V3 full training

Rule of thumb: 2.788M GPU hours is about 57 days if 2048 GPUs are fully utilized

Key efficiency factors:

- Optimized data pipeline
- FP8 training
- Efficient communication protocols
- Auxiliary-loss-free MoE balancing

Hardware Strategy

GPUs used: NVIDIA H800 rather than H100

H800 vs H100:

Feature	H800	H100
Target market	China (export-restricted)	Global
Memory bandwidth	~2TB/s	~3TB/s
Relative capability	Lower than H100	Higher-end part
Cost	Lower	Higher

Why H800 Worked

- MoE lowers active compute relative to a huge dense model
- MLA reduces KV-cache and bandwidth pressure
- Optimized communication and parallelism reduce distributed overhead
- Lower-cost parts may make large clusters easier to afford

Reported cluster size: 2048 H800 GPUs for DeepSeek-V3

Cost Breakdown (Reported)

Commonly cited number: ~$5.6M

- Refers to the reported compute cost for DeepSeek-V3 training
- Does not clearly include total R&D, data curation, experiments, salaries, or earlier failed runs
- So it should not be read as the full cost of building DeepSeek as an organization

Cost Comparison

Compare to frontier models (industry estimates, unverified):

- GPT-4: widely estimated in the tens to hundreds of millions of dollars
- Gemini Ultra: also commonly estimated far above the DeepSeek-reported figure

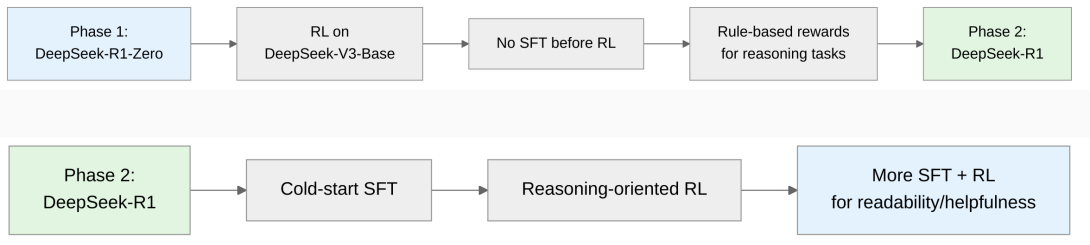
Takeaway: the reported training spend is much lower than many public estimates for frontier training runs

Caveat: exact cost comparisons are weak unless the accounting rules are the same

Reinforcement Learning for Reasoning

DeepSeek-R1 Training Approach

DeepSeek-R1 used a multi-stage post-training pipeline:



Key insight: R1-Zero suggested that strong reasoning behavior can emerge from RL alone, but the final DeepSeek-R1 added SFT stages to improve readability and usability

R1-Zero: RL from Scratch

Training setup:

1. Give model math/coding problems with verifiable answers
2. Model generates a solution attempt
3. Score the result with rule-based rewards when possible
4. Reward correct/format-compliant behavior more than incorrect behavior
5. Update model via RL (Group Relative Policy Optimization)

R1-Zero: Emergent Behaviors

- Self-checking behavior
- Reflection and correction
- Longer reasoning traces
- Backtracking when stuck

Important nuance: this was true for R1-Zero, not for the final DeepSeek-R1 pipeline

Reasoning Quality

Company-reported benchmark snapshot (DeepSeek-R1 vs OpenAI o1-1217):

Benchmark	DeepSeek-R1	OpenAI o1-1217
AIME 2024 (math)	79.8%	79.2%
Codeforces (rating)	2029	2061
GPQA Diamond (science)	71.5%	75.7%

Interpretation: R1 was highly competitive on reasoning benchmarks, but not uniformly better than o1

Reasoning Quality Assessment

- Strong mathematical reasoning
- Competitive coding ability
- Handles multi-step problems well
- Results vary by benchmark and evaluation setup

Why DeepSeek Was Disruptive

Challenging Core Assumptions

Before DeepSeek:

```
Common belief:
- Frontier AI requires $100M+ budgets
- Frontier AI requires the very largest GPU clusters
- Frontier AI requires extensive proprietary data and engineering advantages
```

After DeepSeek (Reported Claims)

```
DeepSeek's reported evidence suggested:
- ~$5-6M training cost for V3 (reported)
- 2048 H800 GPUs for V3 (reported)
- a strong base model + efficient architecture + RL-heavy post-training
- shorter iteration cycles than many observers expected
```

Better takeaway: efficiency and algorithmic choices can materially change the amount of compute needed, even if raw scale still matters

Industry Reaction

Stock market impact (January 27, 2025):

- NVIDIA shares dropped about 17% in one day
- AI hardware stocks broadly sold off
- Investors questioned assumptions about future GPU demand

Why the reaction:

- Suggested that model quality might improve faster than GPU demand assumptions implied
- Open-weight models create more pressure on proprietary businesses
- Raised questions about AI infrastructure investment strategies

Industry Counter-Arguments

- DeepSeek still required major capital, talent, and systems expertise
- The reported cost figure is not the same as total development cost
- Open vs closed model trade-offs remain
- Efficiency and scale may both matter

Strategic Implications

For AI companies:

- Efficiency becomes a more important competitive factor
- MoE architecture gains prominence
- RL-heavy reasoning post-training becomes more prominent
- Pressure on pricing and margins increases

For researchers:

- Strong evidence that architecture and training design matter
- Renewed focus on training efficiency
- Open-weight releases make high-end methods easier to study

Implications for Policy

For policymakers:

- Export controls may be less effective than expected if algorithmic efficiency improves
- AI capability could spread more widely than hardware counts alone suggest
- Regulatory assumptions need to account for rapid efficiency gains

DeepSeek Technical Innovations

Key Optimizations

1. **Multi-Head Latent Attention (MLA):**
   - Compresses KV cache
   - Reduces memory bandwidth requirements
   - Helps make long-context inference more practical

Key Optimizations (Continued)

2. **DeepSeekMoE routing:**
   - Fine-grained expert specialization
   - Better load balancing
   - Auxiliary-loss-free balancing in V3 reduces one common MoE training complication
3. **Training optimizations:**
   - FP8 training
   - Optimized all-to-all communication
   - Pipeline parallelism
   - ZeRO-style optimizer-state sharding

Lessons from DeepSeek

What worked:

- **MoE architecture:** more capacity at similar active compute
- **RL for reasoning:** can induce reasoning behaviors, especially with verifiable rewards
- **Hardware efficiency:** optimize for available hardware
- **Open release:** open weights and technical reports accelerated scrutiny and adoption

What Remains Challenging

- Complex engineering required
- Reproducibility questions remain because some implementation details are missing
- Evaluation benchmark limitations
- Long-term maintenance and updates

Limitations and Open Questions

Known Limitations

Model limitations:

- Reasoning can be verbose or repetitive
- Occasionally makes logical errors
- Limited context window (vs some competitors)
- Training methodology not fully reproducible

Evaluation and Cost Limitations

Evaluation limitations:

- Benchmarks may not capture all capabilities
- Potential overfitting to specific test sets
- Real-world performance vs benchmark performance

Cost reporting:

- Exact training costs not independently verified
- May not include all development costs
- Amortization of failed experiments unclear

Open Questions

Technical questions:

1. Can these techniques scale further (to 10T+ param MoE)?
2. How much does architecture vs data vs training matter?
3. Are there diminishing returns to RL reasoning training?

Economic questions:

1. What is the real total cost of development?
2. How do inference costs compare in production?
3. Is $6M sustainable for future frontier models?

Strategic questions:

1. Will other labs replicate DeepSeek's approach?
2. Does efficiency enable new AI applications?
3. How will this affect AI industry concentration?

Summary

Key Takeaways

Part I: RAG

1. **Embeddings:** Dense vectors capturing semantic meaning
2. **Vector databases:** Efficient similarity search at scale
3. **Retrieval:** Finding relevant context for queries
4. **Generation:** LLM reasoning with grounded context

Key Takeaways (continued)

Part II: ChatGPT Pipeline

1. **Real-time:** Tokenization $\rightarrow$ Transformer $\rightarrow$ Generation $\rightarrow$ Safety
2. **Training:** Pretraining $\rightarrow$ SFT $\rightarrow$ RLHF creates conversational ability
3. **Tools & RAG:** Extend capabilities beyond the base model
4. **Infrastructure:** Optimized serving at massive scale

Key Takeaways (continued)

Part III: Economics

1. Training costs: $10-200M for frontier models
2. Inference dominates at scale
3. Optimization yields 5-10× cost reduction
4. Business models: API, subscription, enterprise

Part IV: DeepSeek

1. MoE: 671B parameters, 37B active per token
2. RL reasoning: Emergent behaviors without labeled data
3. Reported $5-6M cost challenges $100M+ assumptions
4. Industry disruption and strategic implications

Learning Objectives Review

After this lecture, you should be able to:

- Design a RAG pipeline and choose appropriate components
- Trace the end-to-end ChatGPT pipeline from input to output
- Analyze LLM cost structures and business models
- Explain DeepSeek's MoE architecture and RL reasoning approach
- Understand the economic forces shaping the AI industry

References

RAG Resources (clickable links)

Foundational papers:

- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (RAG)
- Dense Passage Retrieval for Open-Domain Question Answering

Tools and frameworks:

- Sentence Transformers
- LlamaIndex
- LangChain RAG guide
- RAGAS evaluation

Vector databases:

- Pinecone
- Qdrant
- Weaviate
- Chroma

ChatGPT and Systems Resources (clickable links)

Training pipelines:

- InstructGPT paper (RLHF)
- GPT-4 Technical Report

System design:

- vLLM: Easy, Fast, and Cheap LLM Serving
- Flash Attention

Safety:

- OpenAI System Card
- Constitutional AI (Anthropic)

Economics Related Resources (clickable links)

Training costs:

- Estimating GPT-3 Training Cost
- Analysis of Large Model Training Costs

Inference economics:

- Inference Cost Analysis
- vLLM benchmarks: https://blog.vllm.ai/

DeepSeek Resources (clickable links)

DeepSeek papers:

- DeepSeek-V3 Technical Report
- DeepSeek-R1 Technical Report
- DeepSeek GitHub: https://github.com/deepseek-ai/

Analysis:

- The Illustrated DeepSeek-R1
- DeepSeek-R1 Recipe Analysis
- DeepSeek Cost Analysis

MoE resources:

- Switch Transformers Paper
- Mixtral MoE